CCE3206 - Digital Signal Processing

Practical 1: The sampling theorem

Name: Dave Galea **Student ID:** 0283404L

Task 1

In [1]:
```
import numpy as np

# simulate for 0.1 s, with 1000 samples
duration = 0.1
N = 1000
# t is a list of 1000 time samples in [0, 0.1)
t = np.linspace(0, duration, N, endpoint=False)

# x is the signal x(t) = sin(2πFt)
x = np.sin(2 * np.pi * 100 * t)

# period of pulse train
train_period = 10
# width of single pulse
pulse_width = 1
# p is a periodic pulse train
p = np.arange(N) % train_period < pulse_width

# y is the sampled sinusoid
y = x * p
```

Task 2

In [2]:
```
from scipy.fft import fft, fftfreq

# F is the list of frequencies for the DFT
F = fftfreq(N, duration / N)

# X(F) is the DFT of x(t)
X = fft(x) / N
# the power spectrum of X(F) is |X(F)|²
Xp = np.abs(X) ** 2

# Repeat for P(F) and Y(F):
P = fft(p) / N
Pp = np.abs(P) ** 2

Y = fft(y) / N
Yp = np.abs(Y) ** 2
```

Task 3

In [3]:
```
import matplotlib.pyplot as plt

# Plot x(t) against t
fig, ax = plt.subplots()
ax.plot(t, x)
ax.set_xlabel('$t$')
ax.set_ylabel('$x(t)$')
```

```
ax.grid(True)

# Repeat for p(t) and y(t)
fig, ax = plt.subplots()
ax.plot(t, p)
ax.set_xlabel('$t$')
ax.set_ylabel('$p(t)$')
ax.grid(True)

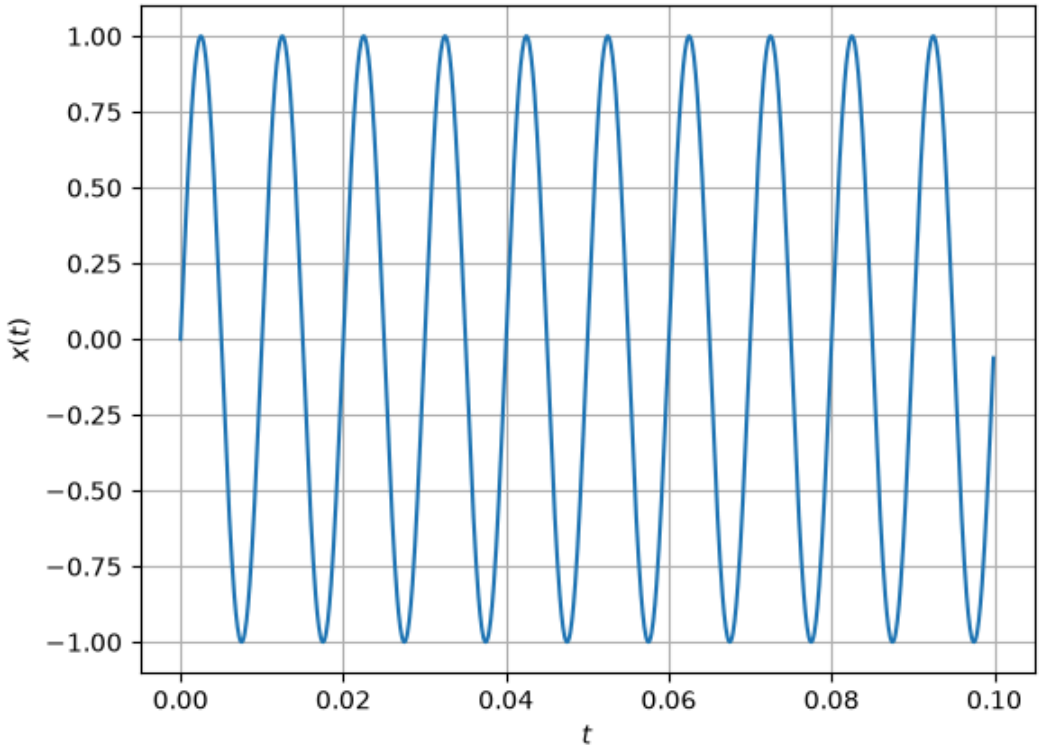
fig, ax = plt.subplots()
ax.plot(t, y)
ax.set_xlabel('$t$')
ax.set_ylabel('$y(t)$')
ax.grid(True)

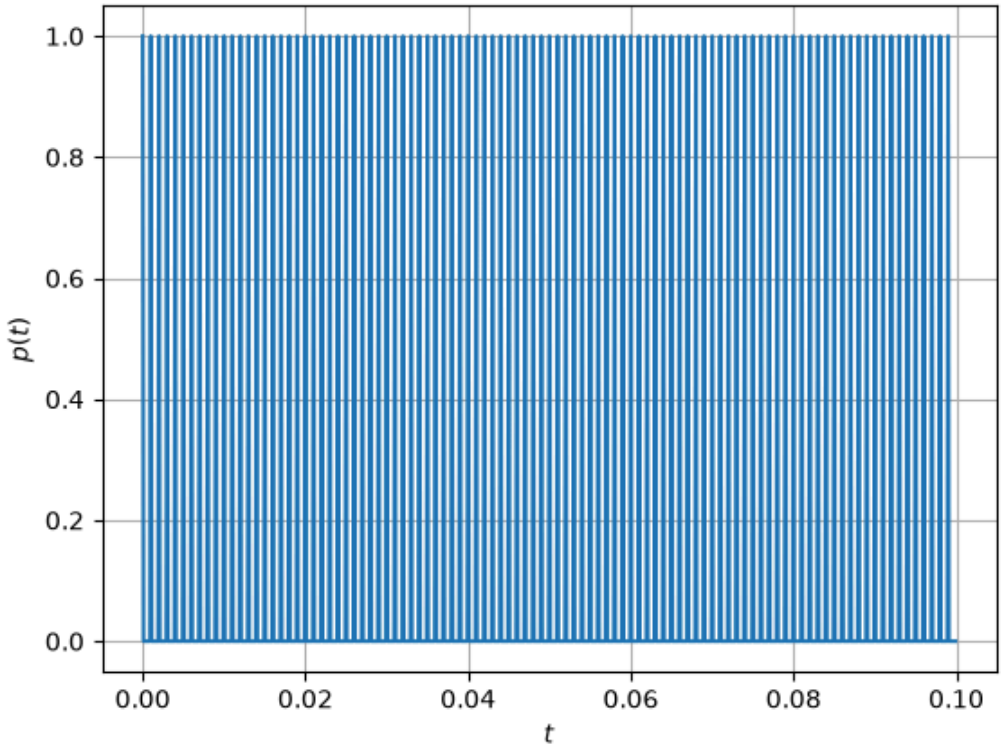
# Plot |X(F)|^2 against F
fig, ax = plt.subplots()
ax.stem(F, Xp)
ax.set_xlabel('$F$')
ax.set_ylabel('$|X(F)|^2$')
ax.grid(True)

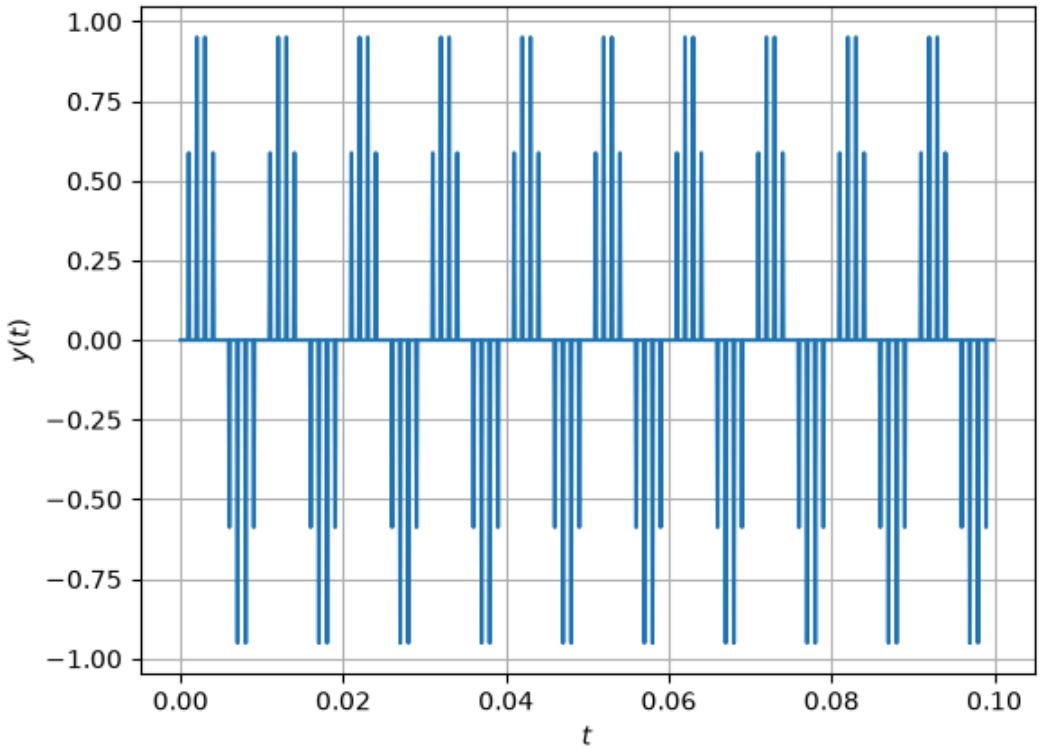
# Repeat for |P(F)|^2 and |Y(F)|^2
fig, ax = plt.subplots()
ax.stem(F, Pp)
ax.set_xlabel('$F$')
ax.set_ylabel('$|P(F)|^2$')
ax.grid(True)

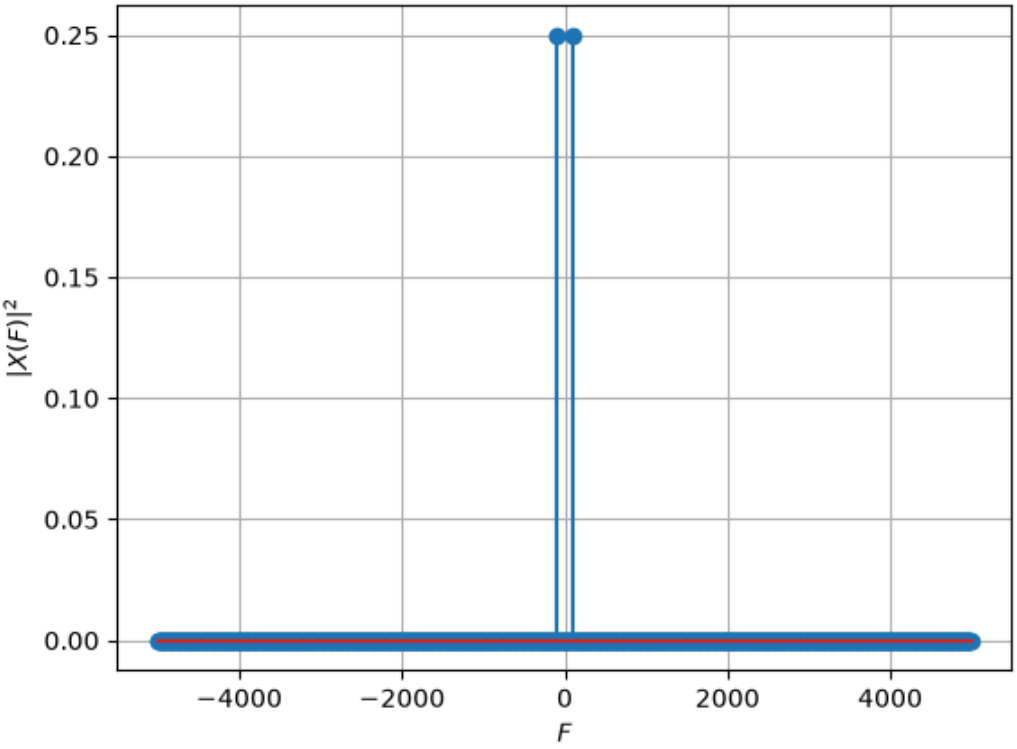
fig, ax = plt.subplots()
ax.stem(F, Yp)
ax.set_xlabel('$F$')
ax.set_ylabel('$|Y(F)|^2$')
ax.grid(True)

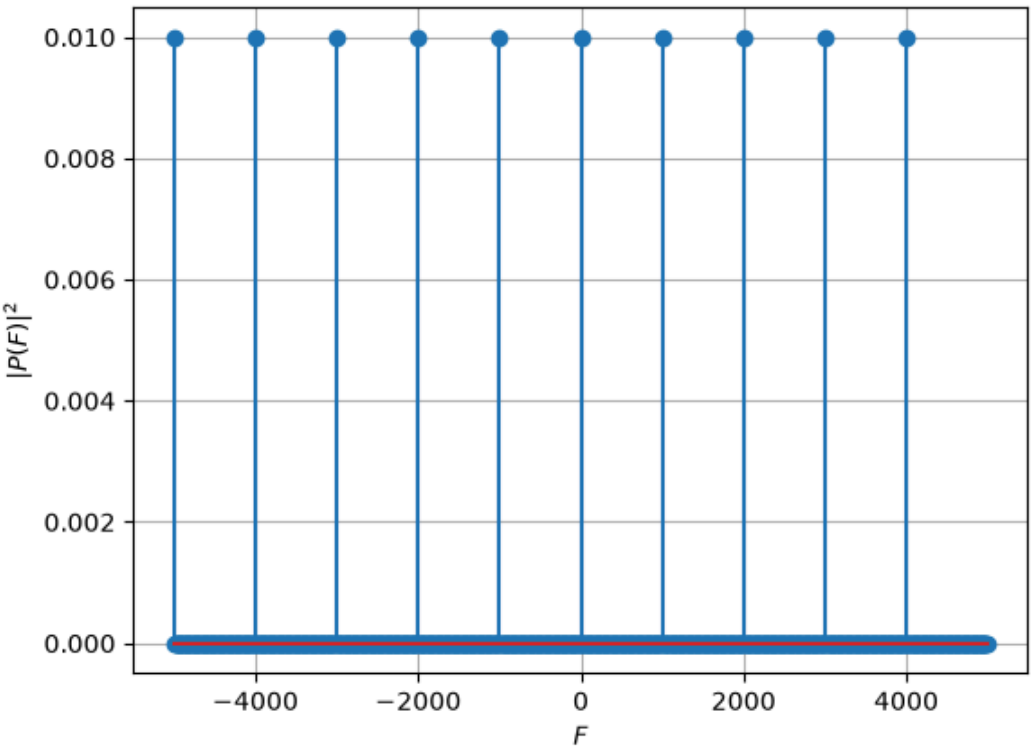
plt.show()
```

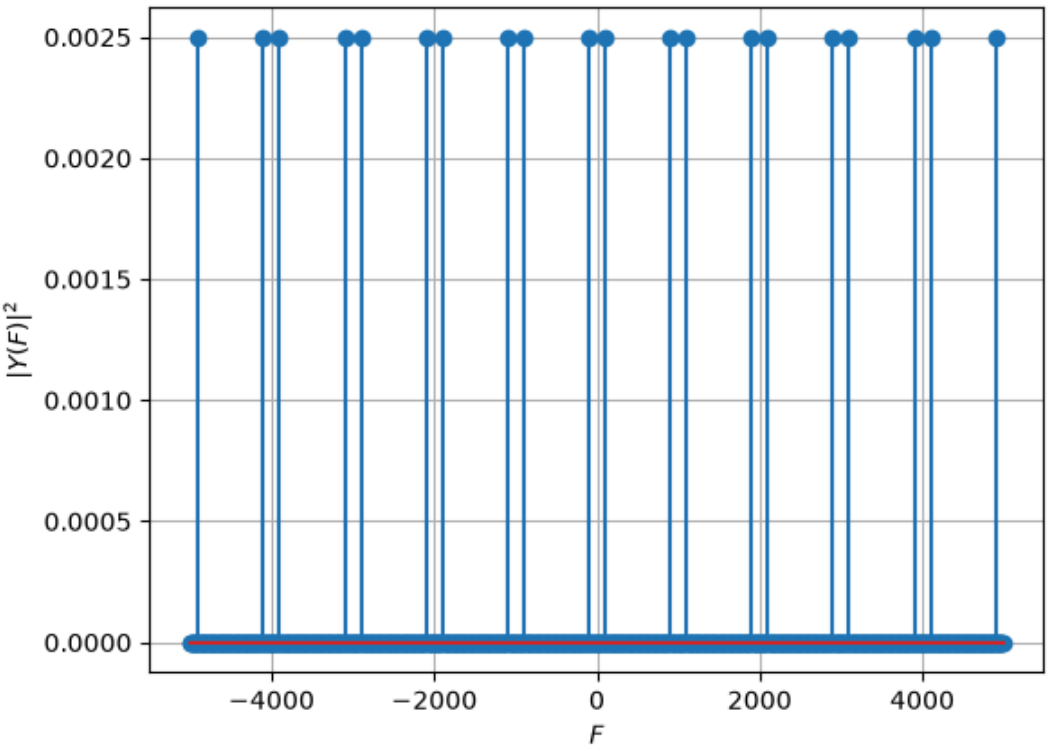



1.0
0.8
0.6
0.4
0.2
0.0
p(t)
0.00
0.02
0.04
0.06
0.08
0.10
t


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
y(t)
0.00
0.02
0.04
0.06
0.08
0.10
t


0.25
0.20
0.15
0.10
0.05
0.00
$|X(F)|^2$
−4000
−2000
0
2000
4000
F


0.010
0.008
0.006
0.004
0.002
0.000
$|P(F)|^2$
−4000
−2000
0
2000
4000
F



Task 4

From the values set for the pulse train:

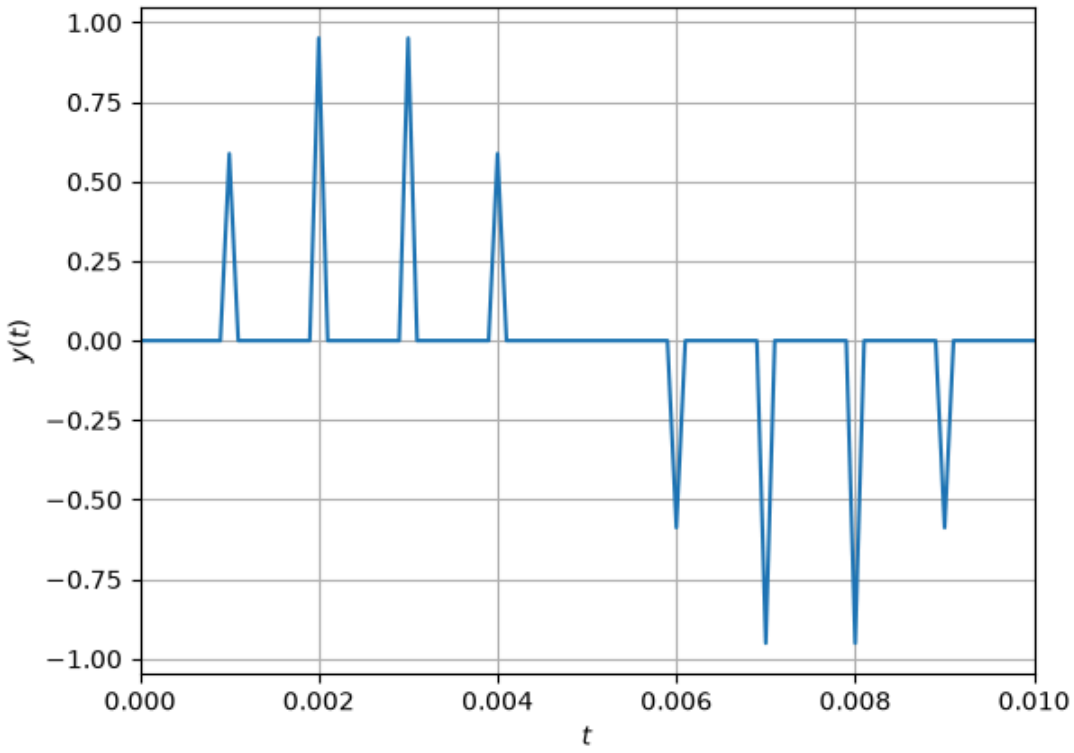
- The simulation sampling frequency is $F_{\text{sim}} = 1000/0.1 = 10{,}000$ Hz, (or 10kHz)
- The pulse train has a period of 10 simulation samples.

Therefore, the effective sampling frequency is:

$$F_s = \frac{10000}{10} = 1000 \text{ Hz}$$

This can be confirmed from the plot of $y(t)$ against t by checking that consecutive sampling instants are present at a rate of 1000 Hz:

In [4]:
```
fig, ax = plt.subplots()
ax.plot(t, y)
ax.set_xlabel('$t$')
ax.set_ylabel('$y(t)$')
ax.grid(True)
ax.set_xlim([0, 0.01])
plt.show()
```



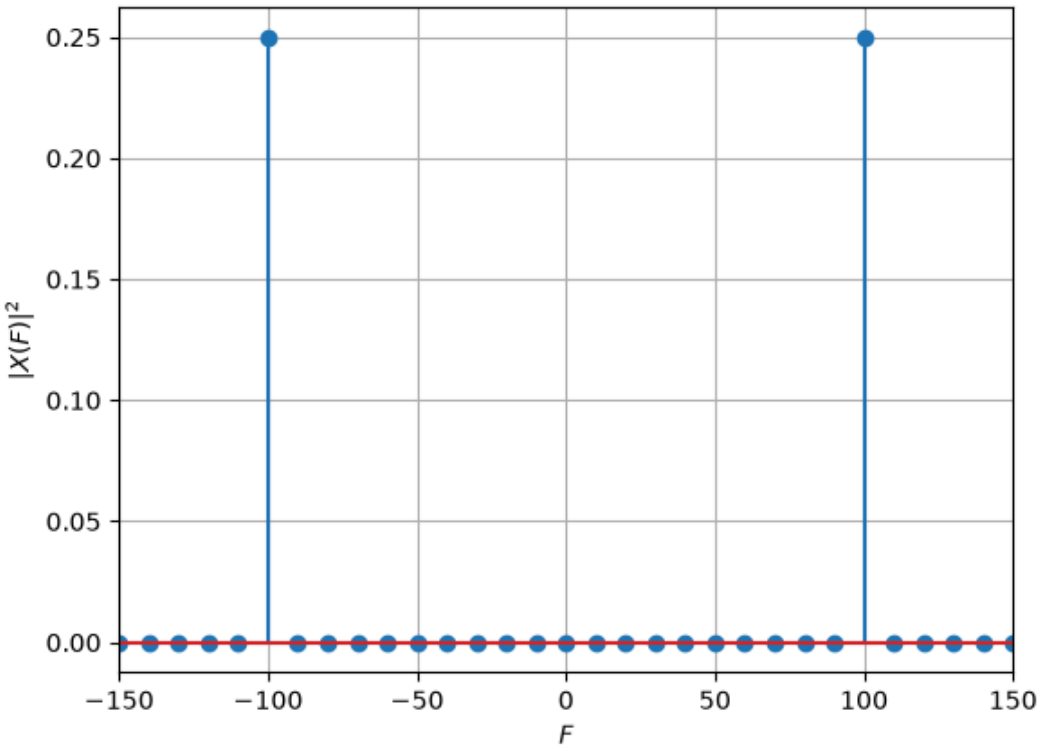
In the interval $0 \leq t < 0.01$ s, there are 10 sampling instants, spaced 0.001 s apart. Only eight spikes are visible because the samples at $t = 0$ s and $t = 0.005$ s have zero amplitude, where the sine wave crosses zero.

Since there are 10 samples in 0.01 seconds, the sampling frequency is $10/0.01 = 1000$ samples per second, confirming the calculated value of 1000 Hz.

Task 5

In [5]:
```
fig, ax = plt.subplots()
ax.stem(F, Xp)
ax.set_xlabel('$F$')
ax.set_ylabel('$|X(F)|^2$')
ax.set_xlim([-150, 150])
ax.grid(True)
plt.show()

for i in range(N):
    if Yp[i] > 0.0001:
        print("Frequency:", F[i], "Hz")
        print(f"Power: {Yp[i]:.4f}")
```




```
Frequency: 100.0 Hz
Power: 0.0025
Frequency: 900.0 Hz
Power: 0.0025
Frequency: 1100.0 Hz
Power: 0.0025
Frequency: 1900.0 Hz
Power: 0.0025
Frequency: 2100.0 Hz
Power: 0.0025
Frequency: 2900.0 Hz
Power: 0.0025
Frequency: 3100.0 Hz
Power: 0.0025
Frequency: 3900.0 Hz
Power: 0.0025
Frequency: 4100.0 Hz
Power: 0.0025
Frequency: 4900.0 Hz
Power: 0.0025
Frequency: -4900.0 Hz
Power: 0.0025
Frequency: -4100.0 Hz
Power: 0.0025
Frequency: -3900.0 Hz
Power: 0.0025
Frequency: -3100.0 Hz
Power: 0.0025
Frequency: -2900.0 Hz
Power: 0.0025
Frequency: -2100.0 Hz
Power: 0.0025
Frequency: -1900.0 Hz
Power: 0.0025
Frequency: -1100.0 Hz
Power: 0.0025
Frequency: -900.0 Hz
Power: 0.0025
Frequency: -100.0 Hz
Power: 0.0025
```

As shown in the graph above, the power spectrum of the unsampled sine wave $x(t)$ has frequency components at -100 Hz and $+100$ Hz, each with power 0.25.

Since the range was quite large for the power spectrum of the sampled sine wave $y(t)$, I coded a simple for loop to print out the frequency components. These are printed above, however for clarity's sake, I repeated them below.

The sampled sine wave $y(t)$ has frequency components at:

± 100, ± 900, ± 1100, ± 1900, ± 2100, ± 2900, ± 3100, ± 3900, ± 4100 and ± 4900 Hz.

Each component has a plotted power of approximately 0.0025. To obtain the corrected power, we multiply by:

$$\left(\frac{F_{\text{sim}}}{F_s}\right)^2 = \left(\frac{10000}{1000}\right)^2 = 100$$

Therefore, the corrected power of each component is:

$$0.0025 \times 100 = 0.25$$

Task 6

Changing frequency and re-running simulation:

In [6]:
```
# Change the sine wave to 800 Hz
x = np.sin(2 * np.pi * 800 * t)

# Sample the new sine wave
y = x * p

# Recalculate the power spectra
X = fft(x) / N
Xp = np.abs(X) ** 2

Y = fft(y) / N
Yp = np.abs(Y) ** 2


# Repeat task 5 :

# Time domain plots:
fig, ax = plt.subplots()
ax.plot(t, x)
ax.set_xlabel('$t$')
ax.set_ylabel('$x(t)$')
ax.grid(True)
ax.set_xlim([0, 0.01])
plt.show()

fig, ax = plt.subplots()
ax.plot(t, y)
ax.set_xlabel('$t$')
ax.set_ylabel('$y(t)$')
ax.set_xlim([0, 0.01])
ax.grid(True)
plt.show()

fig, ax = plt.subplots()
ax.plot(t, x)
ax.set_xlabel('$t$')
```

```
ax.set_ylabel('$x(t)$')
ax.grid(True)
plt.show()

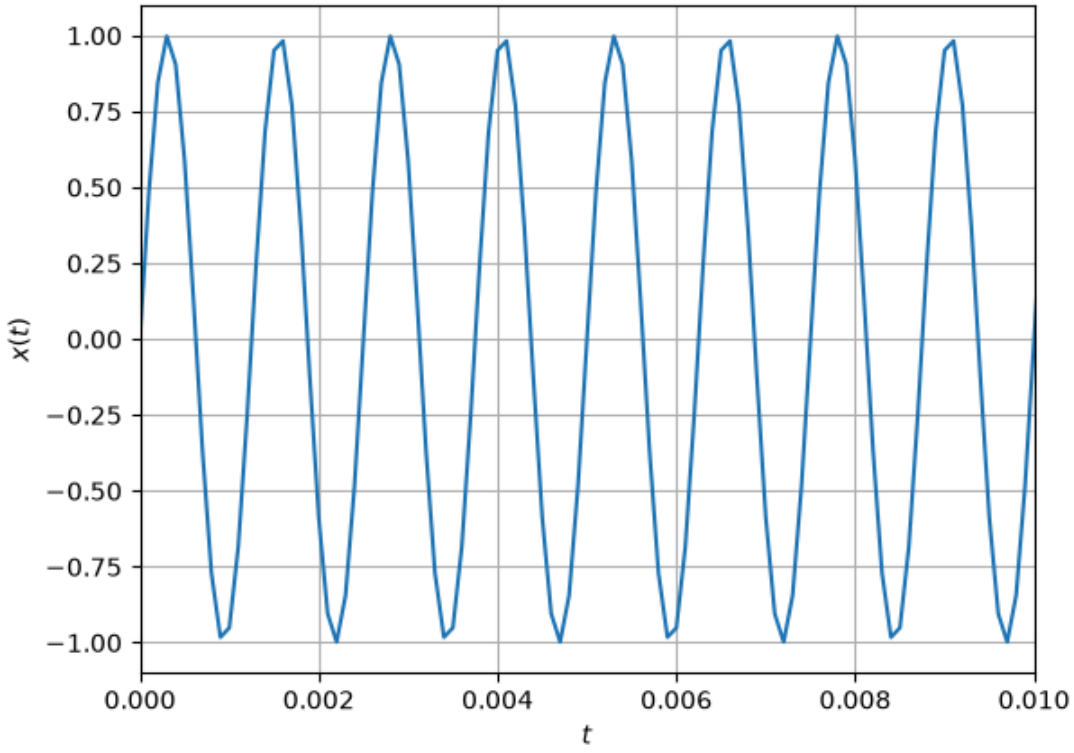
fig, ax = plt.subplots()
ax.plot(t, y)
ax.set_xlabel('$t$')
ax.set_ylabel('$y(t)$')
ax.grid(True)
plt.show()
# Power spectrum plots:
fig, ax = plt.subplots()
ax.stem(F, Yp)
ax.set_xlabel('$F$')
ax.set_ylabel('$|Y(F)|^2$')
ax.grid(True)
plt.show()

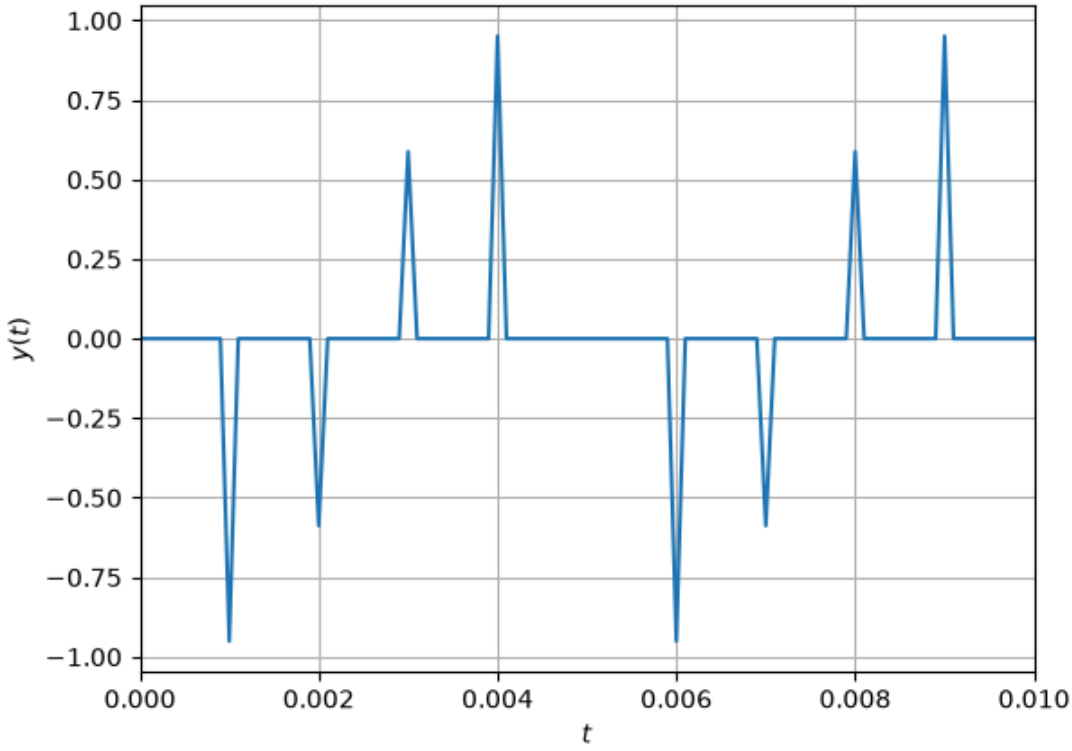
fig, ax = plt.subplots()
ax.stem(F, Xp)
ax.set_xlabel('$F$')
ax.set_ylabel('$|X(F)|^2$')
ax.grid(True)
plt.show()

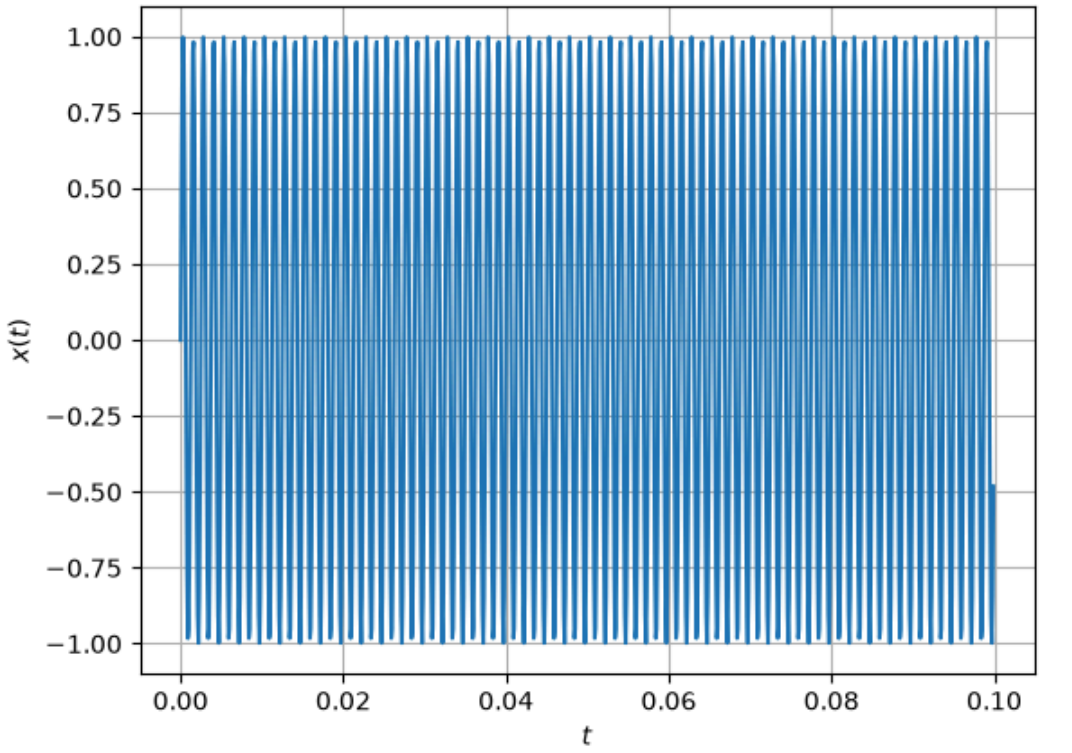
for i in range(N):
    if Xp[i] > 0.0001:
        print("Frequency:", F[i], "Hz")
        print(f"Power: {Xp[i]:.4f}")

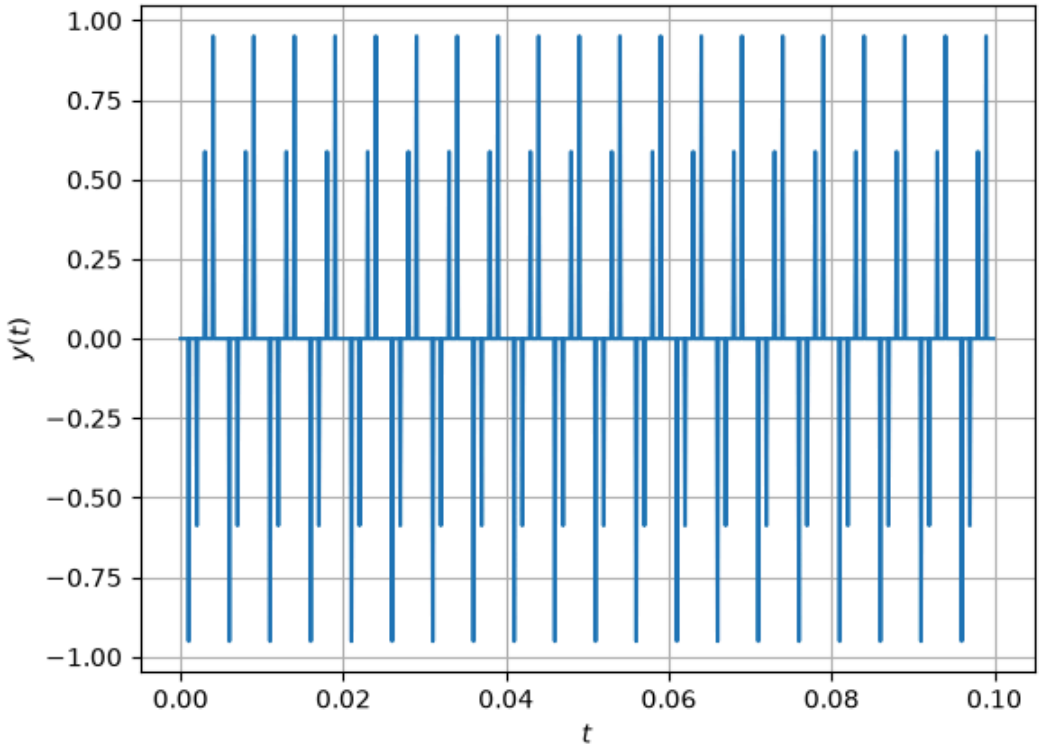
print("==============")

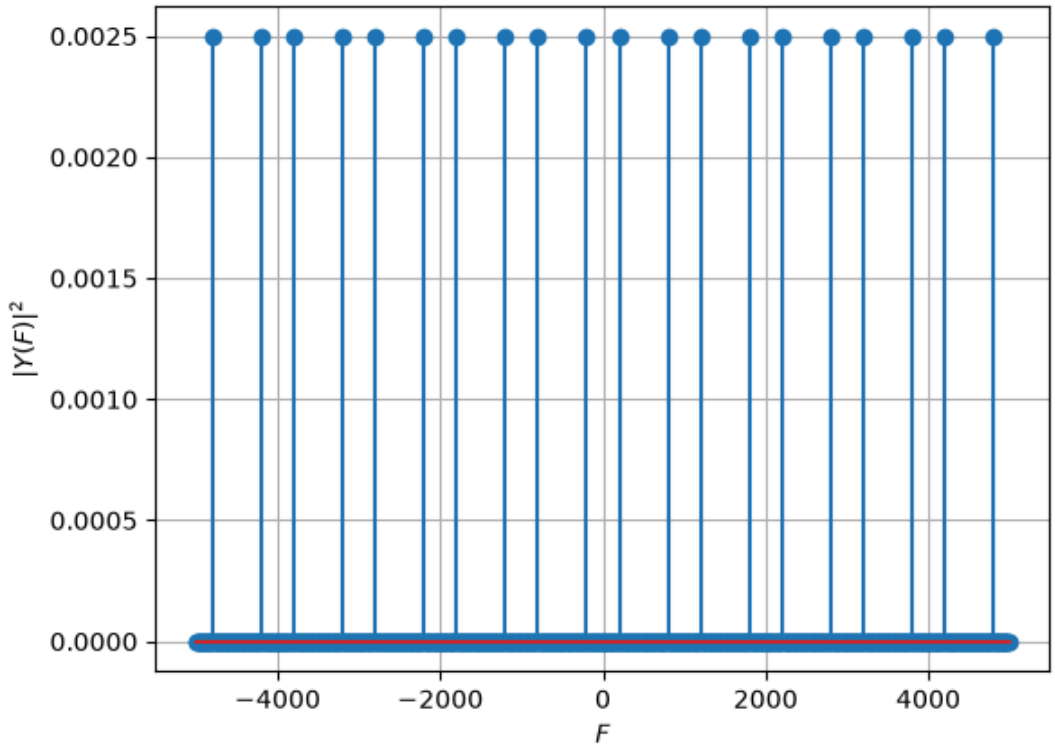
for i in range(N):
    if Yp[i] > 0.0001:
        print("Frequency:", F[i], "Hz")
        print(f"Power: {Yp[i]:.4f}")
```

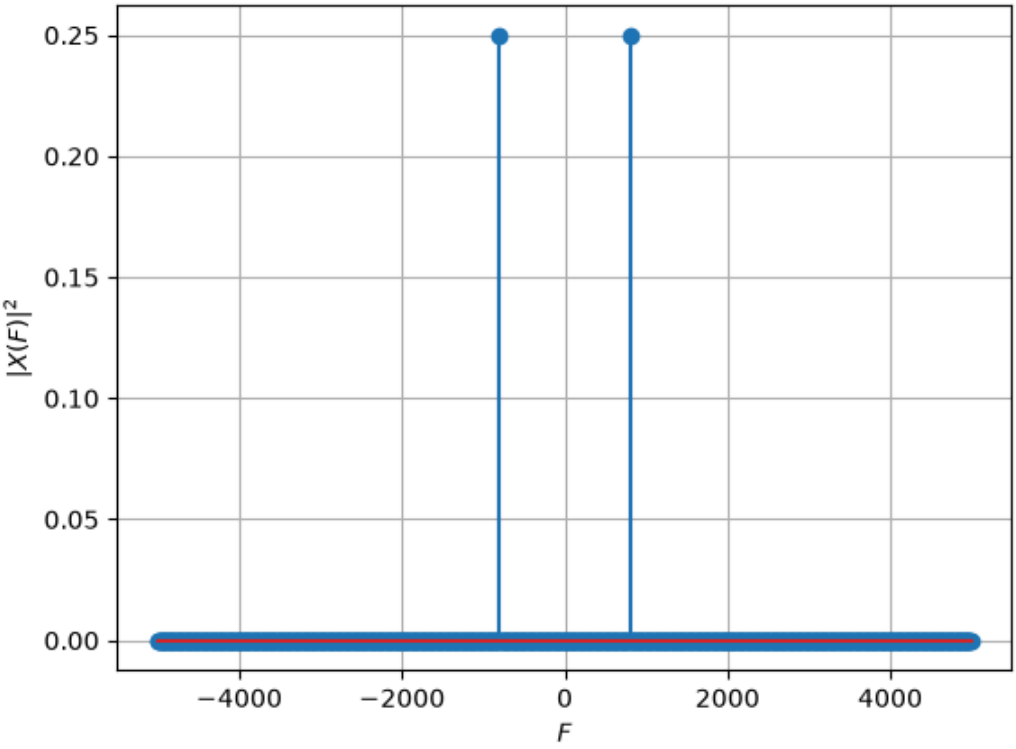



1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
y(t)
0.000
0.002
0.004
0.006
0.008
0.010
t


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
x(t)
0.00
0.02
0.04
0.06
0.08
0.10
t


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
y(t)
0.00
0.02
0.04
0.06
0.08
0.10
t


0.0025
0.0020
0.0015
0.0010
0.0005
0.0000
|Y(F)|²
−4000
−2000
0
2000
4000
F


0.25
0.20
0.15
0.10
0.05
0.00
$|X(F)|^2$
−4000
−2000
0
2000
4000
F

```
Frequency: 800.0 Hz
Power: 0.2500
Frequency: -800.0 Hz
Power: 0.2500
==============
Frequency: 200.0 Hz
Power: 0.0025
Frequency: 800.0 Hz
Power: 0.0025
Frequency: 1200.0 Hz
Power: 0.0025
Frequency: 1800.0 Hz
Power: 0.0025
Frequency: 2200.0 Hz
Power: 0.0025
Frequency: 2800.0 Hz
Power: 0.0025
Frequency: 3200.0 Hz
Power: 0.0025
Frequency: 3800.0 Hz
Power: 0.0025
Frequency: 4200.0 Hz
Power: 0.0025
Frequency: 4800.0 Hz
Power: 0.0025
Frequency: -4800.0 Hz
Power: 0.0025
Frequency: -4200.0 Hz
Power: 0.0025
Frequency: -3800.0 Hz
Power: 0.0025
Frequency: -3200.0 Hz
Power: 0.0025
Frequency: -2800.0 Hz
Power: 0.0025
Frequency: -2200.0 Hz
Power: 0.0025
Frequency: -1800.0 Hz
Power: 0.0025
Frequency: -1200.0 Hz
Power: 0.0025
Frequency: -800.0 Hz
Power: 0.0025
Frequency: -200.0 Hz
Power: 0.0025
```

Repeating of task 5:

The unsampled sine wave $x(t)$ has frequency components at ± 800 Hz, each with power 0.25

The sampled sine wave $y(t)$ has frequency components at:

± 200, ± 800, ± 1200, ± 1800, ± 2200, ± 2800, ± 3200, ± 3800, ± 4200 and ± 4800 Hz.

Each component has a plotted power of approximately 0.0025. To obtain the corrected power, we multiply by:

$$\left(\frac{F_{\text{sim}}}{F_s}\right)^2 = \left(\frac{10000}{1000}\right)^2 = 100$$

Therefore, the corrected power of each component is:

$$0.0025 \times 100 = 0.25$$

Comments:

What changed in this re-run simulation is the frequency of the sine wave from $100Hz$ to $800Hz$; the sampling frequency remained the same ($1000Hz$).

The Nyquist frequency is half the sampling rate, therefore $500Hz$; since the frequency of the sine wave is higher than the Nyquist frequency, **aliasing** occurs. This means that from the samples, the original signal cannot be reconstructed. This is exemplified by the plots below this markdown cell.

In comparison, the original 100 Hz sine wave was below the Nyquist frequency, so aliasing did not occur.

In [7]:
```
fig, ax = plt.subplots()

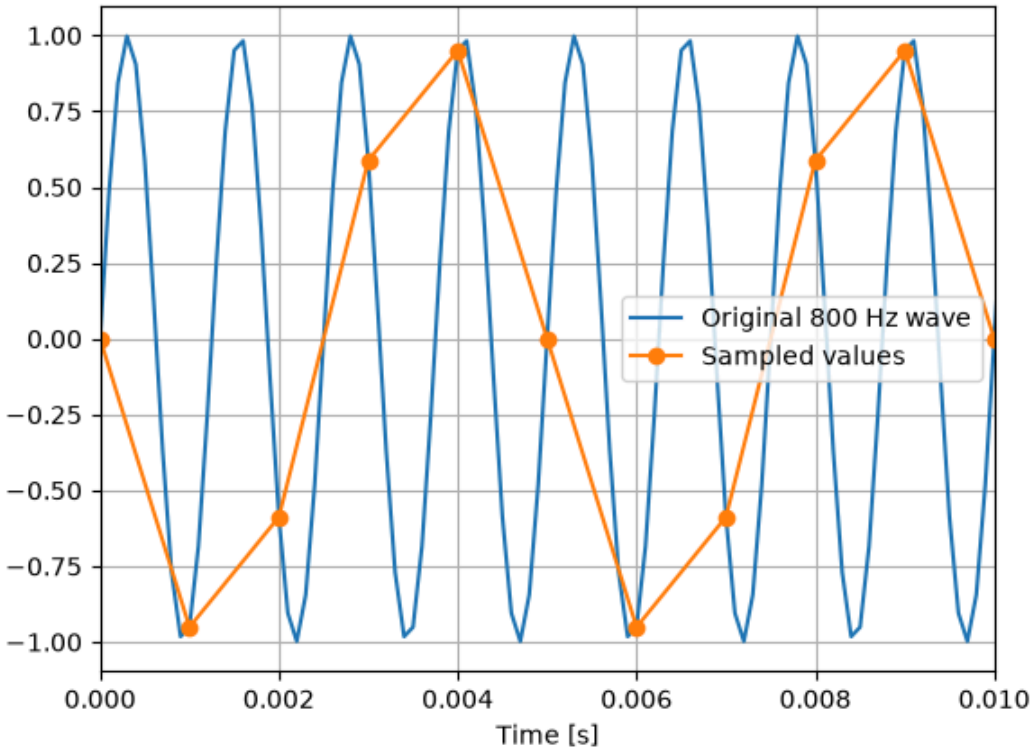
ax.plot(t, x, label='Original 800 Hz wave')
ax.plot(t[p], y[p], 'o-', label='Sampled values')

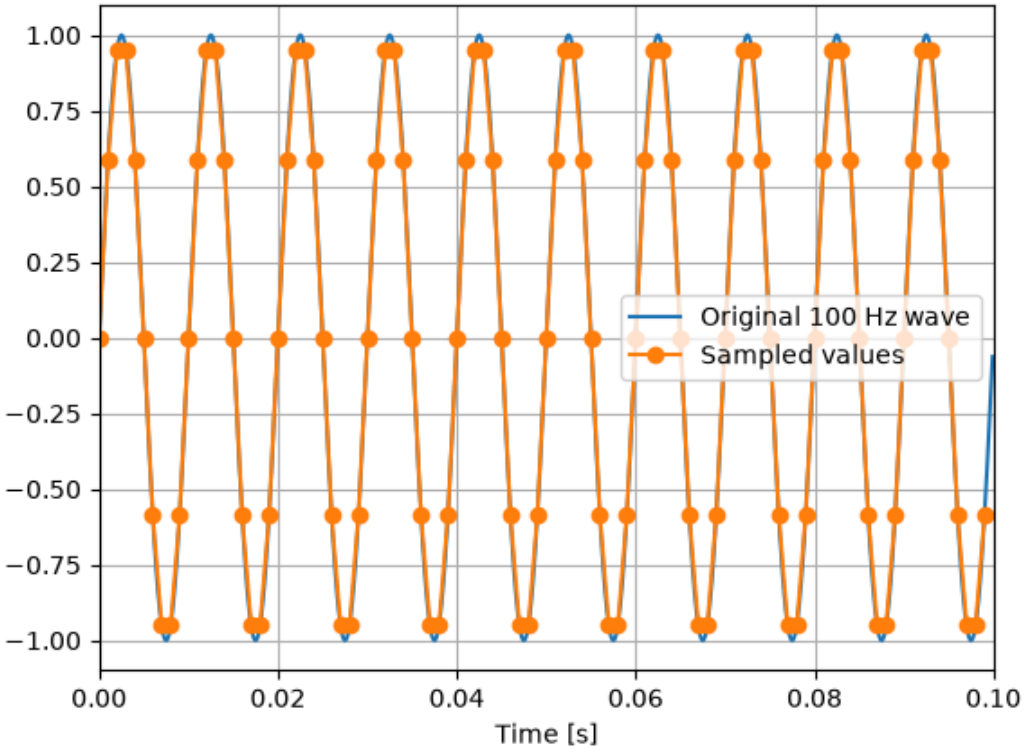
ax.set_xlim([0, 0.01])
ax.set_xlabel('Time [s]')
ax.legend()
ax.grid(True)
plt.show()

x = np.sin(2 * np.pi * 100 * t)
y = x * p

fig, ax = plt.subplots()
ax.plot(t, x, label='Original 100 Hz wave')
ax.plot(t[p], y[p], 'o-', label='Sampled values')

ax.set_xlim([0, 0.1])
ax.set_xlabel('Time [s]')
ax.legend()
ax.grid(True)
plt.show()
```





Task 7

In [8]:

```
import matplotlib.pyplot as plt
# Return to a 100 Hz sine wave
x = np.sin(2 * np.pi * 100 * t)

# Increase the pulse width to 4 samples
pulse_width = 4
p = np.arange(N) % train_period < pulse_width

y = x * p

# Recalculate a power spectra
X = fft(x) / N
Xp = np.abs(X) ** 2

P = fft(p) / N
Pp = np.abs(P) ** 2

Y = fft(y) / N
Yp = np.abs(Y) ** 2


# Plot x(t) against t
fig, ax = plt.subplots()
ax.plot(t, x)
ax.set_xlabel('$t$')
ax.set_ylabel('$x(t)$')
ax.grid(True)

# Repeat for p(t) and y(t)
fig, ax = plt.subplots()
ax.plot(t, p)
ax.set_xlabel('$t$')
ax.set_ylabel('$p(t)$')
ax.grid(True)

fig, ax = plt.subplots()
ax.plot(t, y)
```

```
ax.set_xlabel('$t$')
ax.set_ylabel('$y(t)$')
ax.grid(True)

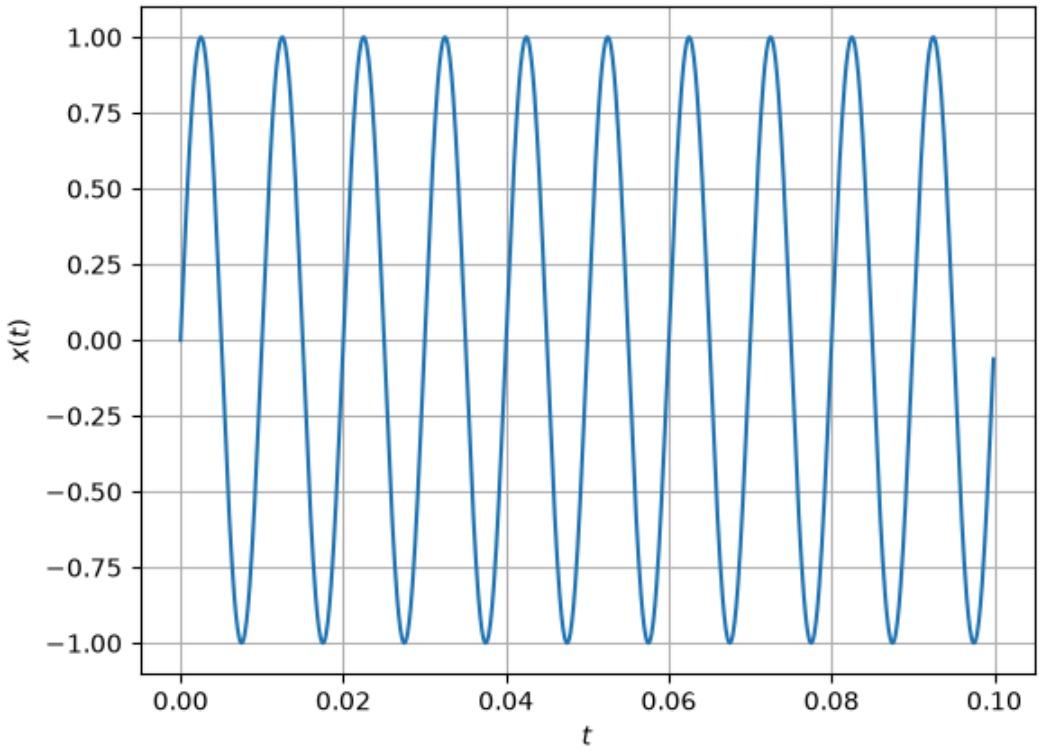
# Plot |X(F)|^2 against F
fig, ax = plt.subplots()
ax.stem(F, Xp)
ax.set_xlabel('$F$')
ax.set_ylabel('$|X(F)|^2$')
ax.grid(True)

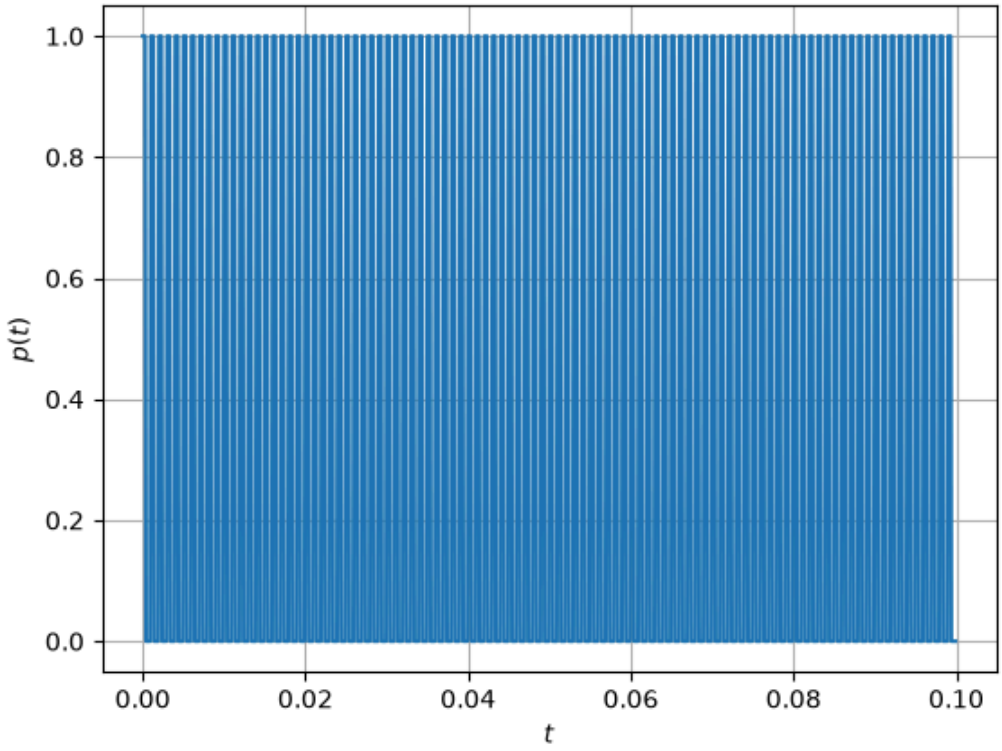
# Repeat for |P(F)|^2 and |Y(F)|^2
fig, ax = plt.subplots()
ax.stem(F, Pp)
ax.set_xlabel('$F$')
ax.set_ylabel('$|P(F)|^2$')
ax.grid(True)

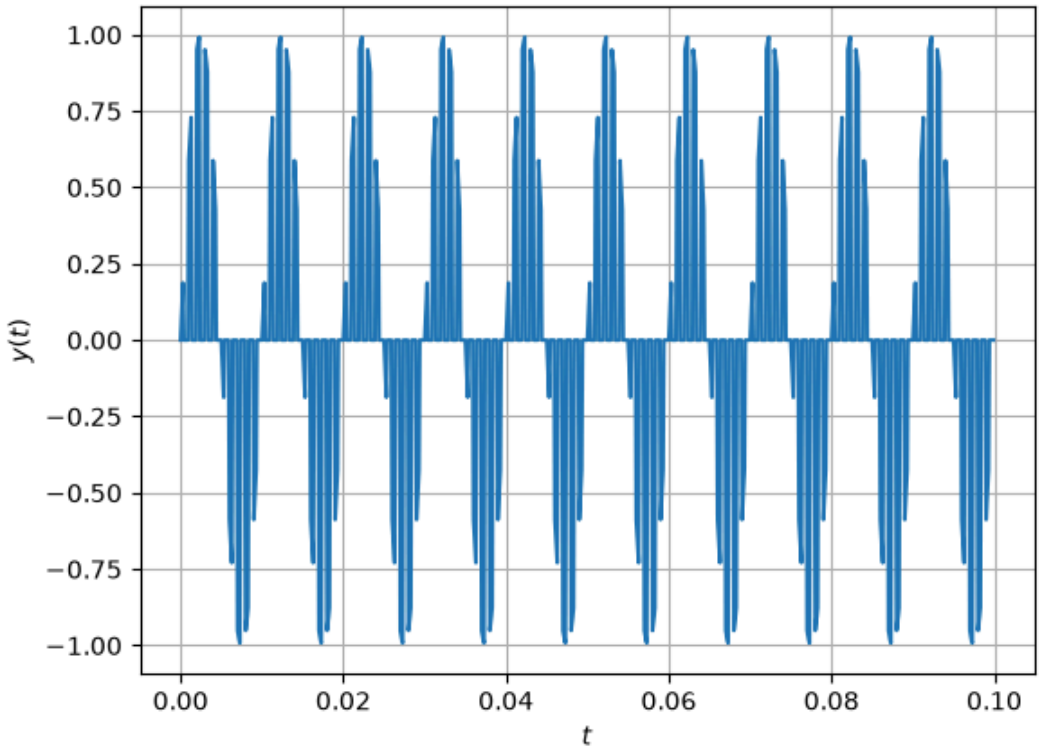
fig, ax = plt.subplots()
ax.stem(F, Yp)
ax.set_xlabel('$F$')
ax.set_ylabel('$|Y(F)|^2$')
ax.grid(True)

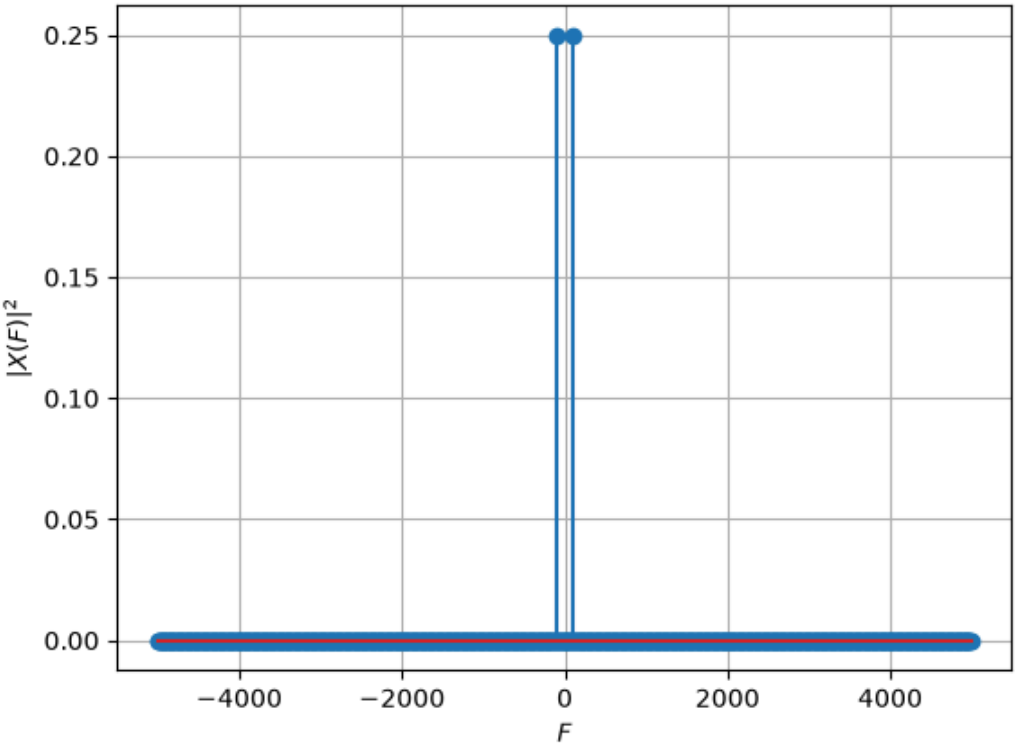
plt.show()

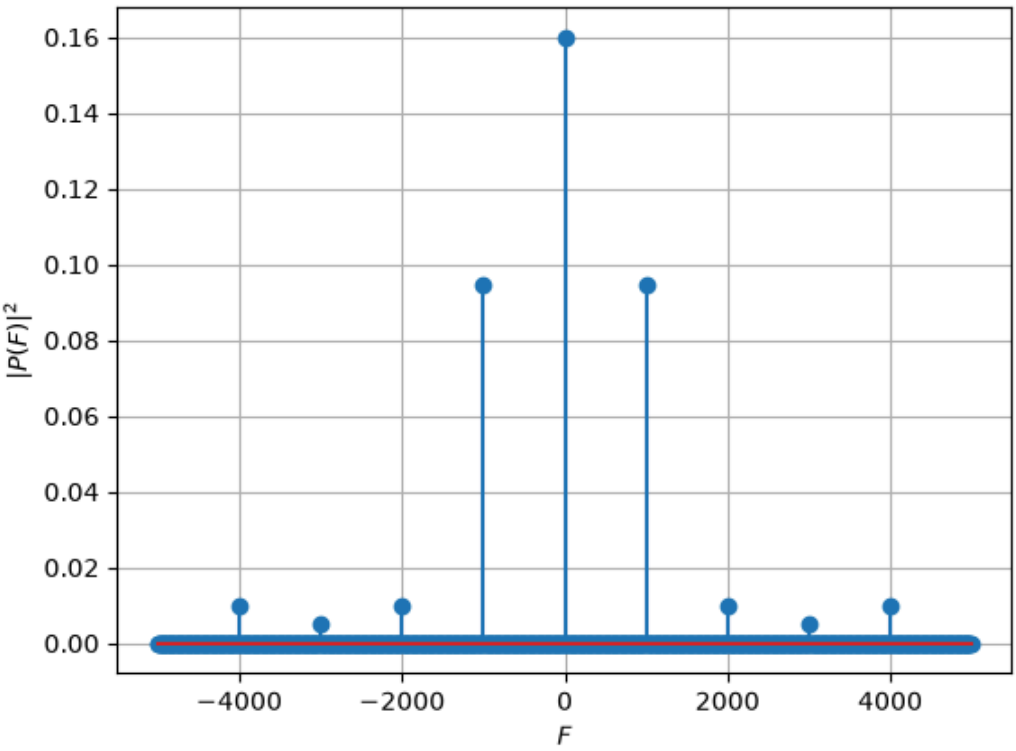
for i in range(N):
    if Yp[i] > 0.000001:
        print("Frequency:", F[i], "Hz")
        print(f"Power: {Yp[i]:.6f}")
```

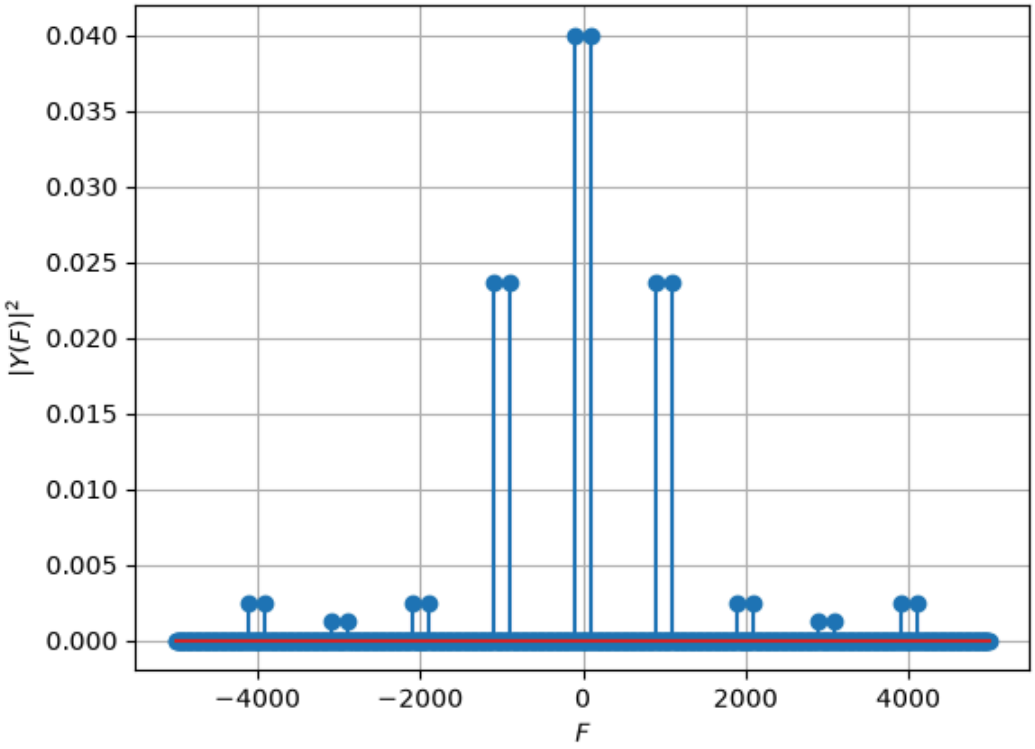



1.0
0.8
0.6
0.4
0.2
0.0
p(t)
0.00
0.02
0.04
0.06
0.08
0.10
t


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
y(t)
0.00
0.02
0.04
0.06
0.08
0.10
t


0.25
0.20
0.15
0.10
0.05
0.00
$|X(F)|^2$
−4000
−2000
0
2000
4000
F


0.16
0.14
0.12
0.10
0.08
0.06
0.04
0.02
0.00
$|P(F)|^2$
−4000
−2000
0
2000
4000
F




```
Frequency: 100.0 Hz
Power: 0.040000
Frequency: 900.0 Hz
Power: 0.023680
Frequency: 1100.0 Hz
Power: 0.023680
Frequency: 1900.0 Hz
Power: 0.002500
Frequency: 2100.0 Hz
Power: 0.002500
Frequency: 2900.0 Hz
Power: 0.001320
Frequency: 3100.0 Hz
Power: 0.001320
Frequency: 3900.0 Hz
Power: 0.002500
Frequency: 4100.0 Hz
Power: 0.002500
Frequency: -4100.0 Hz
Power: 0.002500
Frequency: -3900.0 Hz
Power: 0.002500
Frequency: -3100.0 Hz
Power: 0.001320
Frequency: -2900.0 Hz
Power: 0.001320
Frequency: -2100.0 Hz
Power: 0.002500
Frequency: -1900.0 Hz
Power: 0.002500
Frequency: -1100.0 Hz
Power: 0.023680
Frequency: -900.0 Hz
Power: 0.023680
Frequency: -100.0 Hz
Power: 0.040000
```

Repeating of task 5:

The unsampled sine wave $x(t)$ has frequency components at ± 100 Hz, each with power 0.25.

The sampled sine wave $y(t)$ has frequency components at:

± 100, ± 900, ± 1100, ± 1900, ± 2100, ± 2900, ± 3100, ± 3900 and ± 4100 Hz.

The components have different plotted powers. To obtain the corrected powers, we multiply by:

$$\left(\frac{F_{\text{sim}}}{F_s}\right)^2 = \left(\frac{10000}{1000}\right)^2 = 100$$

Therefore, the measured and corrected powers are:

Frequency components (Hz)	Measured power per component	Corrected power per component
± 100	0.040000	4.000000
± 900, ± 1100	0.023680	2.368034
± 1900, ± 2100	0.002500	0.250000
± 2900, ± 3100	0.001320	0.131966
± 3900, ± 4100	0.002500	0.250000

Comments:

Increasing the pulse width from 1 to 4 samples keeps each pulse on for four times longer. Therefore, $y(t)$ contains wider sections of the original sine wave, increasing the average power.

The highest measured power of $y(t)$ increased from 0.0025 to 0.04, at $\pm 100 Hz$. Since changing pulse width also changes how power is distributed, the frequency components now have different powers, unlike the one sample pulse case where they all had power 0.0025. The peaks at ± 4900 Hz also practically disappear.

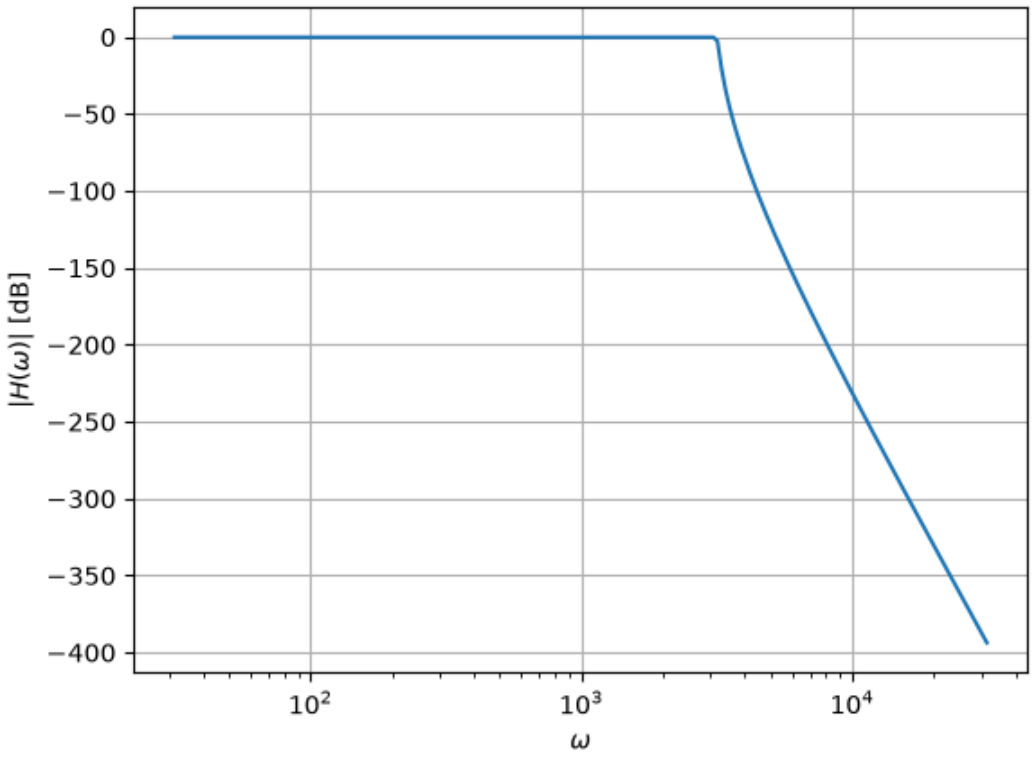
Task 8

In [9]:
```
from scipy import signal

# passband cut-off frequency
wp = 2 * np.pi * 500
# stopband edge frequency
ws = 2 * np.pi * 600
# maximum loss in passband (dB)
rp = 0.1
# minimum attenuation in stopband (dB)
rs = 60

# determine the order of the Chebyshev filter
[n, wn] = signal.cheb1ord(
    wp, ws, rp, rs, analog=True
)
# determine the transfer function
[bnorm, anorm] = signal.cheby1(n, rp, 1, analog=True)
# determine the response of the filter
[w, h] = signal.freqs(bnorm, anorm)

# plot the response
fig, ax = plt.subplots()
ax.semilogx(wn * w, 20 * np.log10(abs(h )))
ax.set_xlabel('$\\omega$')
ax.set_ylabel('$|H(\\omega)|$ [dB]')
ax.grid(True)
plt.show()
```



Task 9

In [10]:
```
# Change the sine wave to 400 Hz
x = np.sin(2 * np.pi * 400 * t)

# Reset the pulse width to 1 sample
pulse_width = 1
p = np.arange(N) % train_period < pulse_width

# Recalculate the sampled signal
y = x * p
```

In [11]:
```
# simulate the filter operation
tr, r, state = signal.lsim((bnorm, anorm), U=y, T=wn * t)

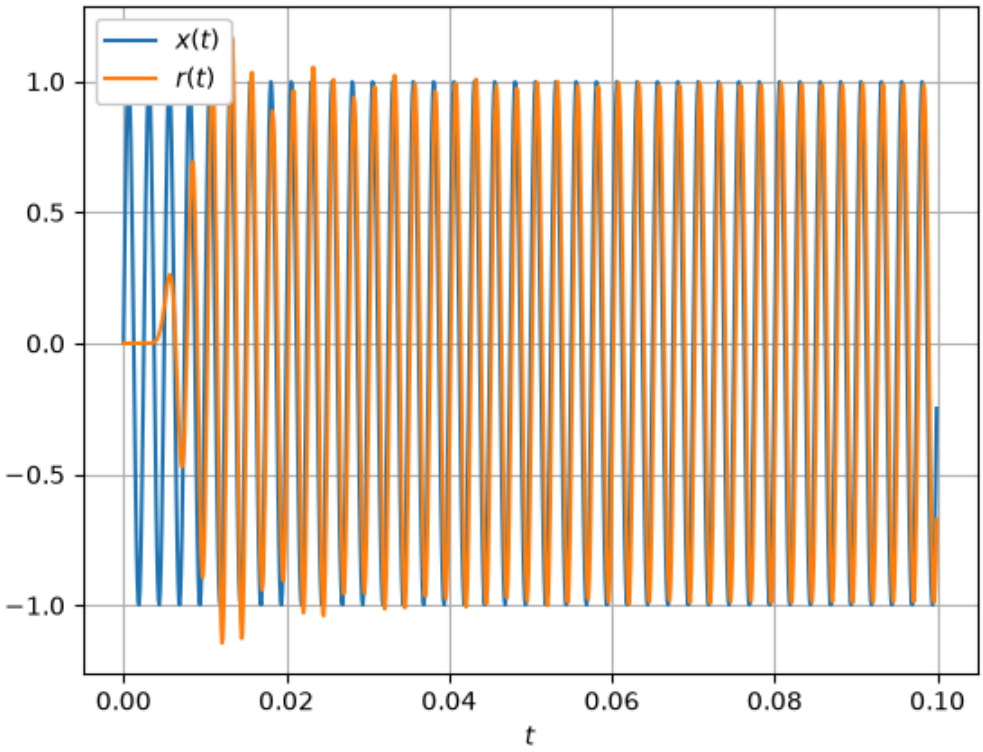
# plot x(t) and r(t)
fig, ax = plt.subplots()
ax.plot(t, x, label='$x(t)$')
ax.plot(t, train_period * r, label='$r(t)$')
ax.set_xlabel('$t$')
ax.legend(loc='best', framealpha=1)
ax.grid(True)
plt.show()

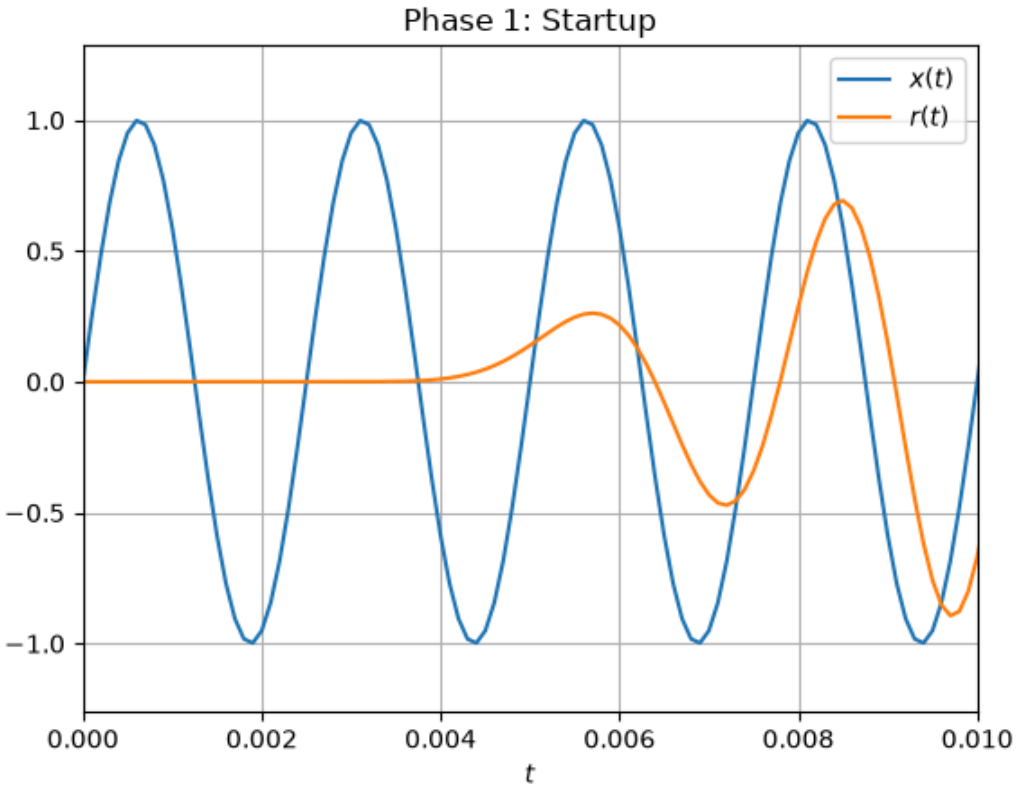
# Phase 1: startup
fig, ax = plt.subplots()
ax.plot(t, x, label='$x(t)$')
ax.plot(t, train_period * r, label='$r(t)$')
ax.set_xlim([0, 0.01])
ax.set_title('Phase 1: Startup')
ax.set_xlabel('$t$')
ax.legend()
ax.grid(True)
plt.show()

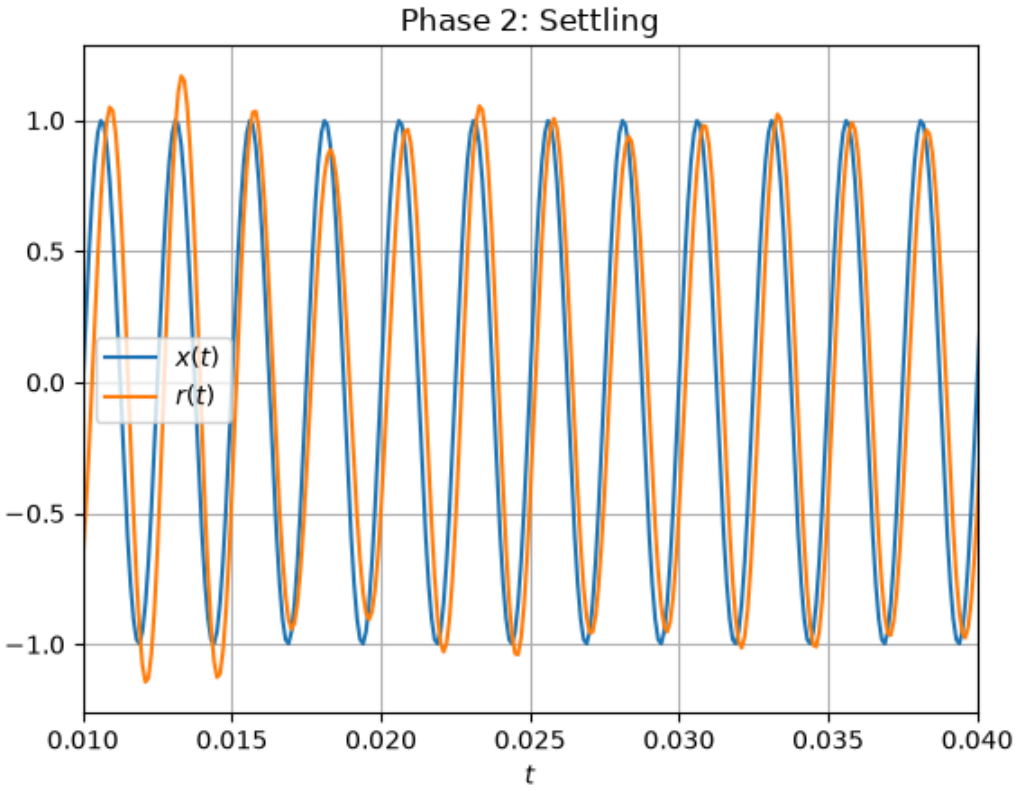
# Phase 2: settling
fig, ax = plt.subplots()
ax.plot(t, x, label='$x(t)$')
```

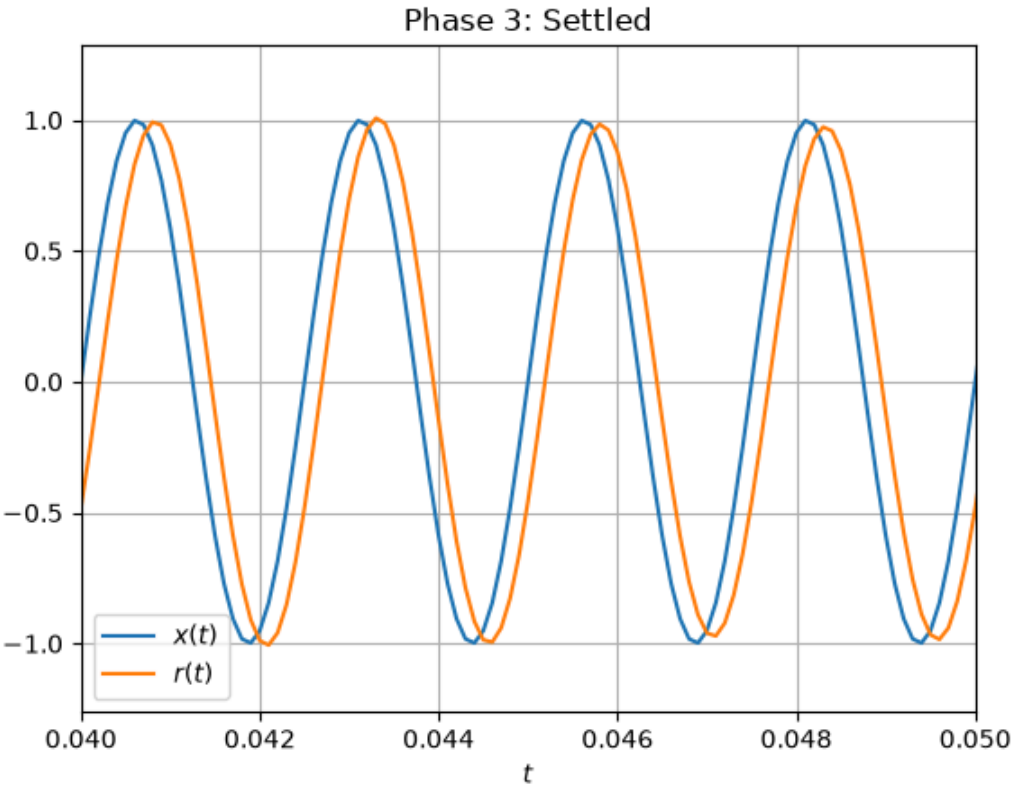
```
ax.plot(t, train_period * r, label='$r(t)$')
ax.set_xlim([0.01, 0.04])
ax.set_title('Phase 2: Settling')
ax.set_xlabel('$t$')
ax.legend()
ax.grid(True)
plt.show()

# Phase 3: settled
fig, ax = plt.subplots()
ax.plot(t, x, label='$x(t)$')
ax.plot(t, train_period * r, label='$r(t)$')
ax.set_xlim([0.04, 0.05])
ax.set_title('Phase 3: Settled')
ax.set_xlabel('$t$')
ax.legend()
ax.grid(True)
plt.show()
```




Phase 1: Startup
x(t)
r(t)
1.0
0.5
0.0
−0.5
−1.0
0.000
0.002
0.004
0.006
0.008
0.010
t


Phase 2: Settling
x(t)
r(t)
1.0
0.5
0.0
−0.5
−1.0
0.010
0.015
0.020
0.025
0.030
0.035
0.040
t



Comments:

During startup, the reconstructed signal $r(t)$ is initially close to zero before its amplitude begins to increase.

During settling, its amplitude briefly exceeds that of the original signal, then gradually settles to approximately the same amplitude. It's amplitude also sometimes dips below that of the original signal at times.

Once settled, $r(t)$ has approximately the same amplitude and frequency as the original 400 Hz sine wave. However, the peaks do not line up exactly, showing that the filter introduces a phase shift.

Since 400 Hz is within the filter's passband of 500 Hz, the original sine wave is able to be recovered; albeit with a phase shift.

Task 10

In [12]:
```
# Change the sine wave to 600 Hz
x = np.sin(2 * np.pi * 600 * t)

# Reset the pulse width to 1 sample
pulse_width = 1
p = np.arange(N) % train_period < pulse_width

# Recalculate the sampled signal
y = x * p

# Simulate the filter operation
tr, r, state = signal.lsim((bnorm, anorm), U=y, T=wn * t)

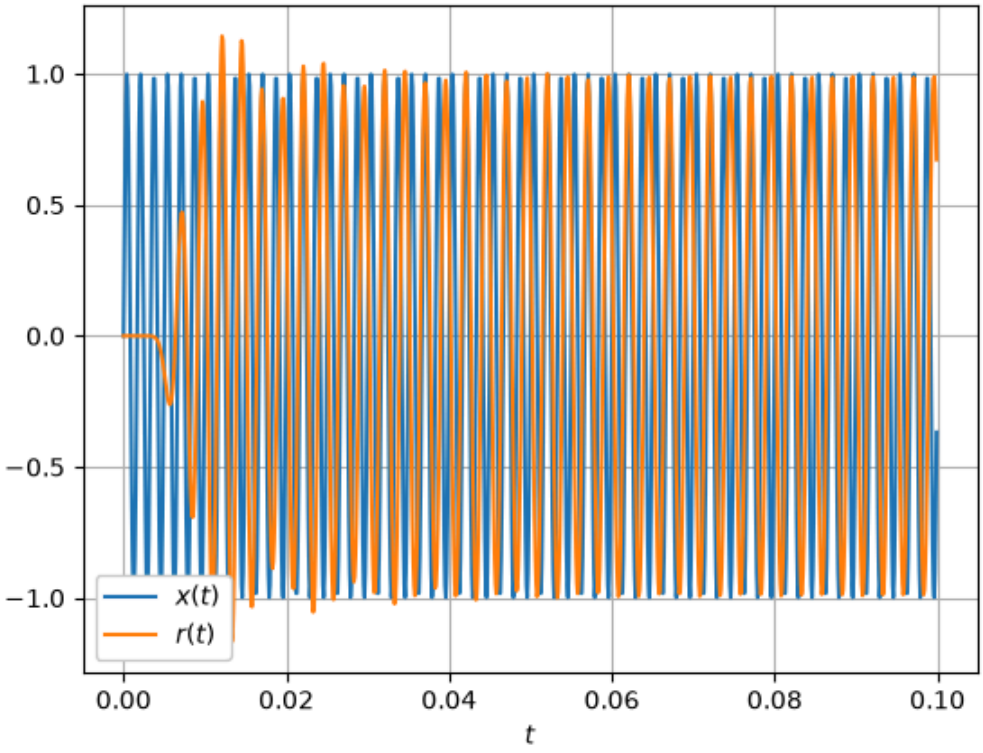
# Plot x(t) and r(t)
fig, ax = plt.subplots()
ax.plot(t, x, label='$x(t)$')
ax.plot(t, train_period * r, label='$r(t)$')
ax.set_xlabel('$t$')
```

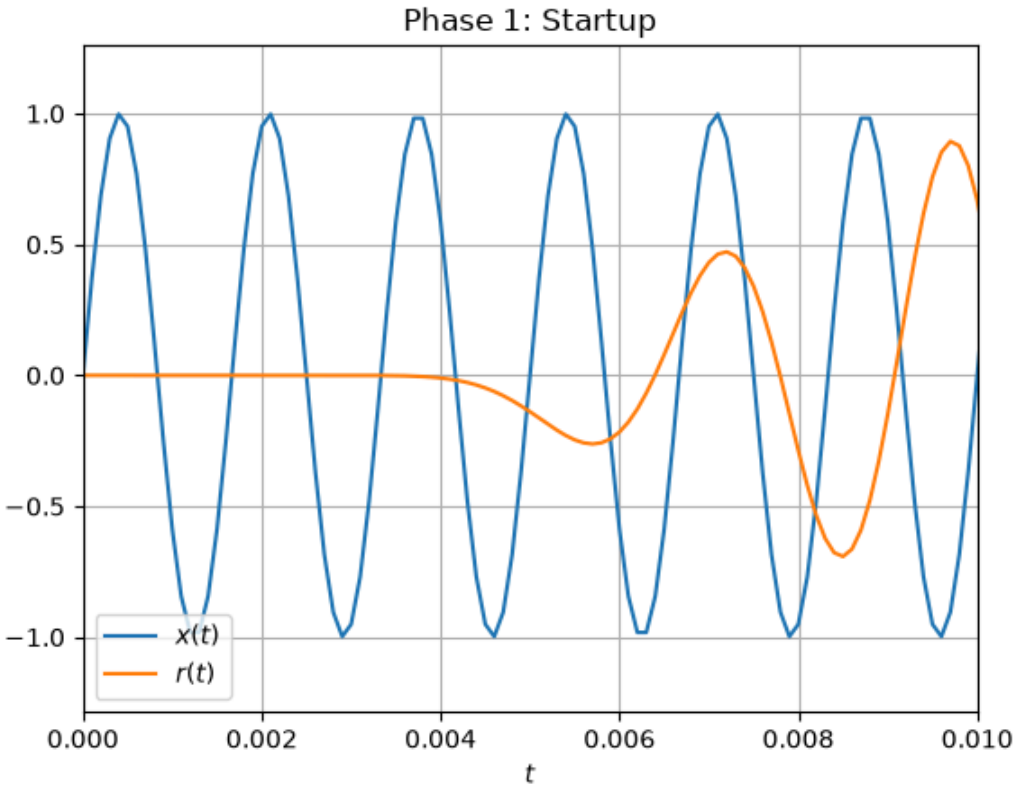
```
ax.legend(loc='best', framealpha=1)
ax.grid(True)
plt.show()

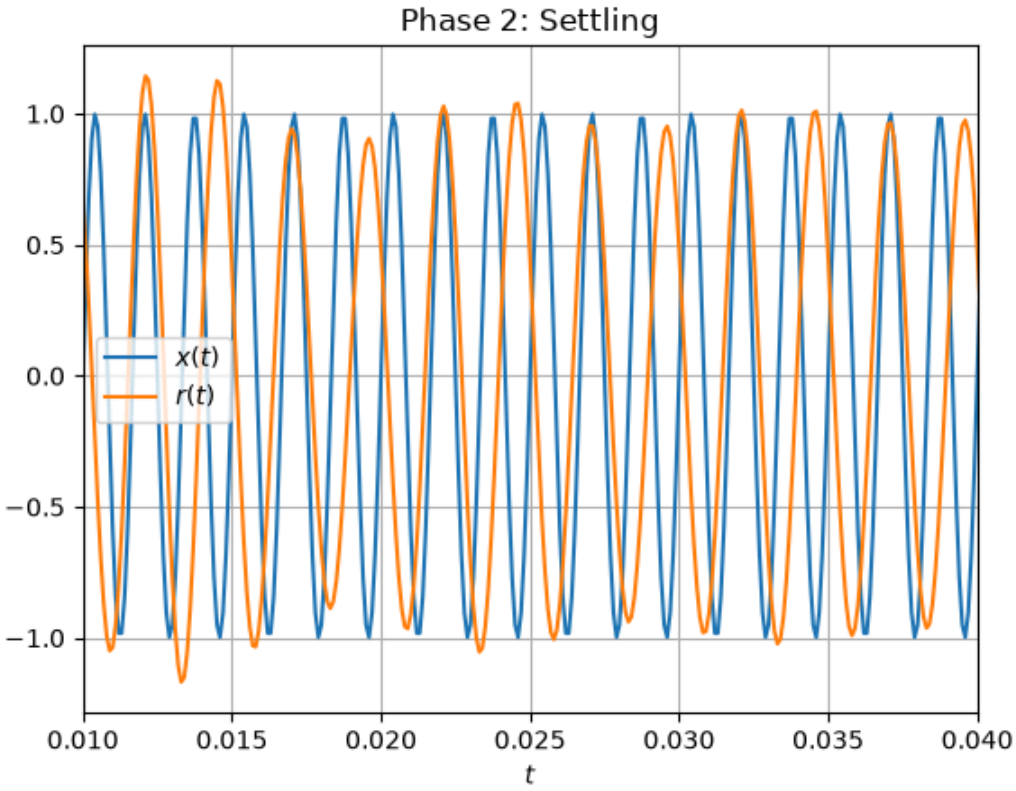
# Phase 1: startup
fig, ax = plt.subplots()
ax.plot(t, x, label='$x(t)$')
ax.plot(t, train_period * r, label='$r(t)$')
ax.set_xlim([0, 0.01])
ax.set_title('Phase 1: Startup')
ax.set_xlabel('$t$')
ax.legend()
ax.grid(True)
plt.show()

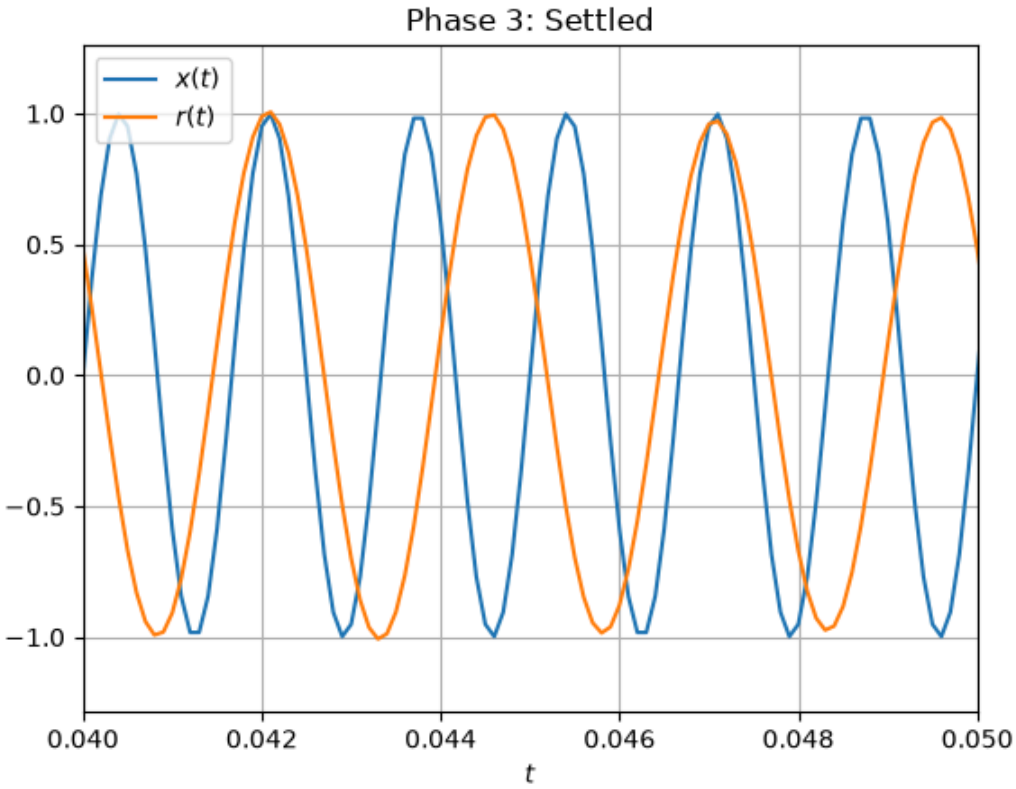
# Phase 2: settling
fig, ax = plt.subplots()
ax.plot(t, x, label='$x(t)$')
ax.plot(t, train_period * r, label='$r(t)$')
ax.set_xlim([0.01, 0.04])
ax.set_title('Phase 2: Settling')
ax.set_xlabel('$t$')
ax.legend()
ax.grid(True)
plt.show()

# Phase 3: Settled
fig, ax = plt.subplots()
ax.plot(t, x, label='$x(t)$')
ax.plot(t, train_period * r, label='$r(t)$')
ax.set_xlim([0.04, 0.05])
ax.set_title('Phase 3: Settled')
ax.set_xlabel('$t$')
ax.legend()
ax.grid(True)
plt.show()
```




Phase 1: Startup
1.0
0.5
0.0
−0.5
−1.0
x(t)
r(t)
0.000
0.002
0.004
0.006
0.008
0.010
t


Phase 2: Settling
1.0
0.5
0.0
−0.5
−1.0
x(t)
r(t)
0.010
0.015
0.020
0.025
0.030
0.035
0.040
t



Comments:

The startup and settling phases are very similar to those of task 9, with initial amplitude of the reconstructed signal close to 0 during startup, and the settling phase having some larger or smaller peaks in amplitude compared to the original signal.

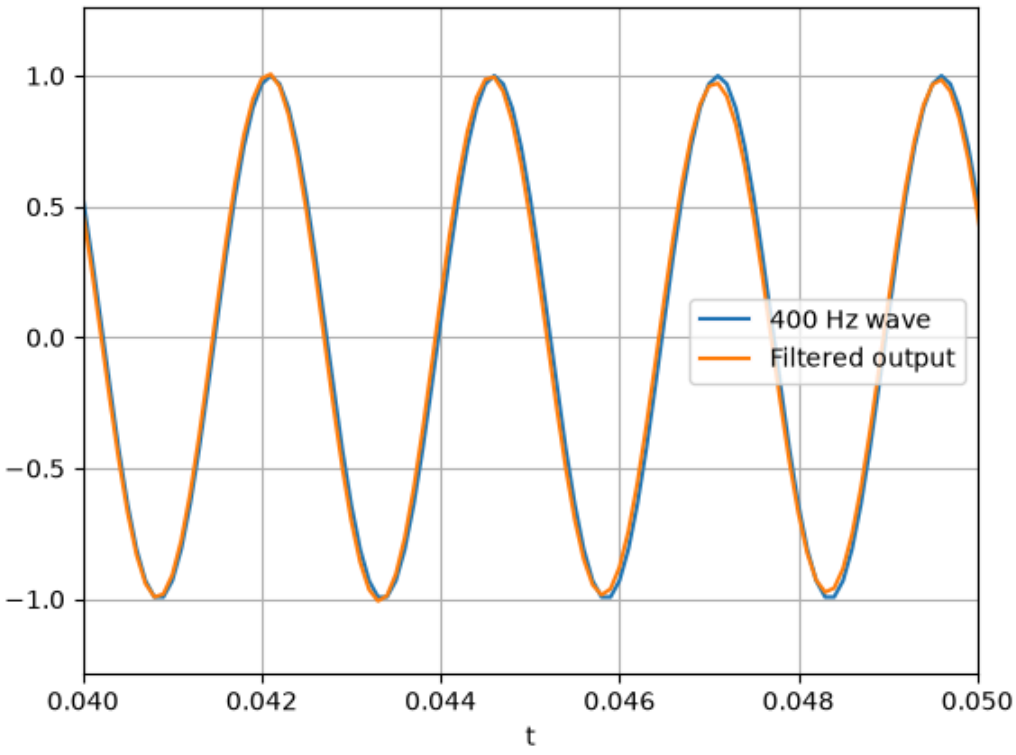
Although the filter in task 9 introduced a phase shift, the original and reconstructed signals oscillated at the same speed; this is not the case for task 10. The output oscillates more slowly than the original 600 Hz sine wave. Sampling at 1000 Hz introduces an aliased component at $1000 - 600 = 400$ Hz.

We can show this by generating a 400 Hz signal, and overlapping it with the filtered signal, to verify similar oscillation frequency. This is done in the graph below this markdown cell. The phase is adjusted so that the peaks align and the focus of the graph goes to the frequency of oscillation.

In Task 9, the filter approximately recovered the original 400 Hz signal. In Task 10, the output is approximately 400 Hz even though the original signal is 600 Hz, so reconstruction is unsuccessful.

In [13]:
```
comparison = np.cos(2 * np.pi * 400 * (t - 0.0421))

fig, ax = plt.subplots()
ax.plot(t, comparison, label='400 Hz wave')
ax.plot(t, train_period * r, label='Filtered output')
ax.set_xlim([0.04, 0.05])
ax.set_xlabel('t')
ax.legend()
ax.grid(True)
plt.show()
```


1.0
0.5
0.0
−0.5
−1.0
0.040
0.042
0.044
0.046
0.048
0.050
t
400 Hz wave
Filtered output